ARCHITECTURE WALKTHROUGH

From Monolith to Microservices

A step-by-step guided tour of the architecture — we build it together, one piece at a time.

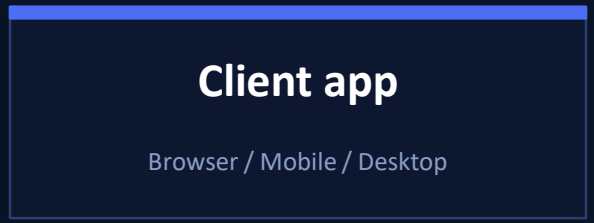
01	02	03	04	05	06	07
The Client	API Gateway	Core Services	Databases	Sync Calls	Async Events	Full Picture

Students: you don't need to know everything yet — we reveal the architecture one layer at a time.

STEP 01 — THE CLIENT

Where does every request start?

Before any service exists — there's a user doing something.



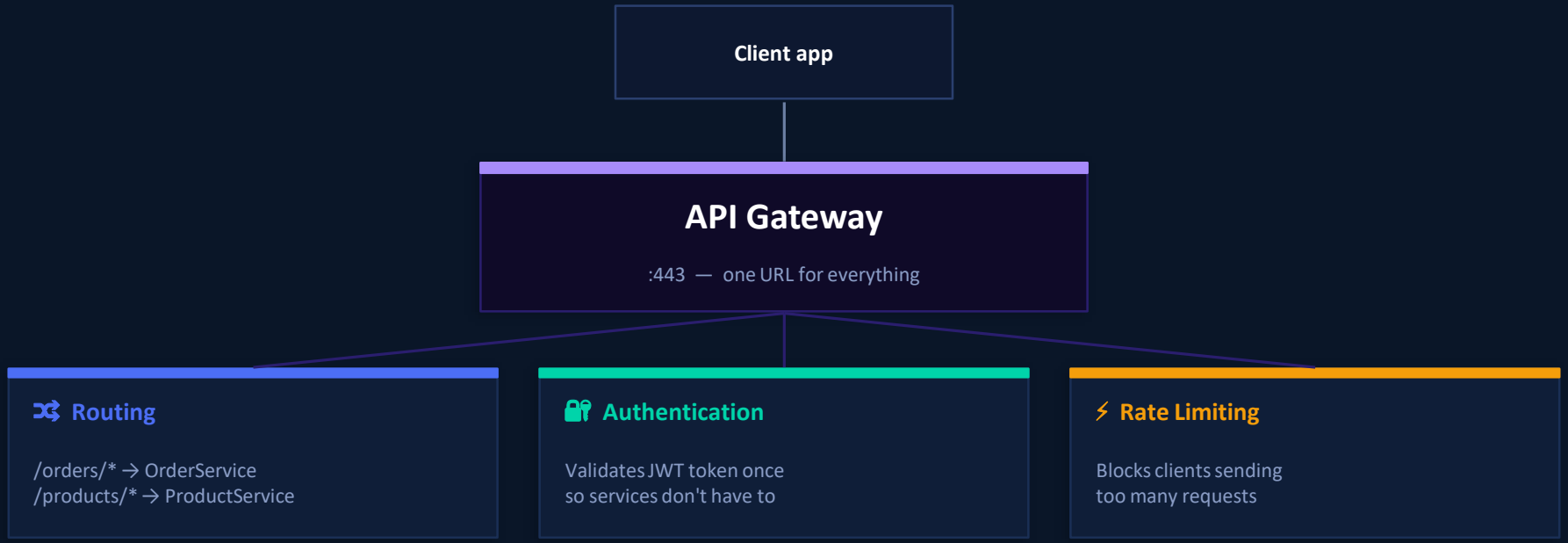
- User taps 'Place Order'
- App sends HTTP request
- Waits for a response

Discussion: The client sends a request — but to WHERE exactly? How does it know which service to call?

STEP 02 — THE API GATEWAY

The single front door

The client only ever knows ONE address. The gateway handles the rest.

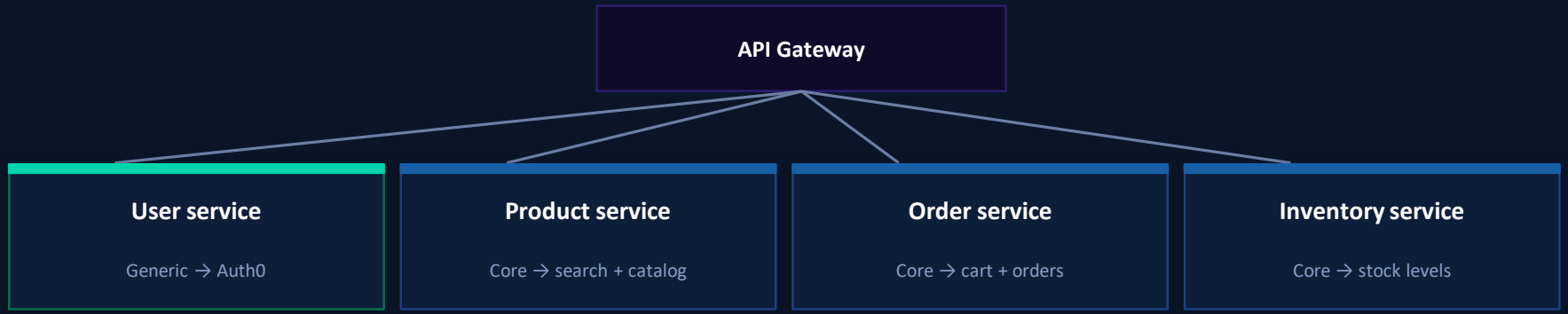


Key insight: services behind the gateway don't need to handle auth themselves — the gateway already did it.

STEP 03 — CORE SERVICES

The services that do the real work

Gateway routes the request — now a service handles it.



What is Generic?

UserService is Generic because login is a solved problem. We use Auth0 — no point building our own.

What is Core?

ProductService, OrderService, InventoryService are Core — they contain YOUR business rules and competitive logic.

One service = one job

Each service is responsible for EXACTLY one domain. It doesn't reach into another service's data.

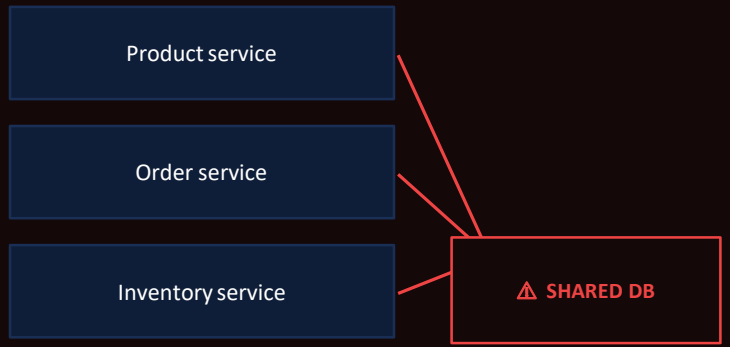
Discussion: Why is UserService Generic but OrderService Core? What's the difference in business value?

STEP 04 — DATA ISOLATION

Every service owns its own database

The golden rule: no two services share a database. Ever.

✕ WRONG

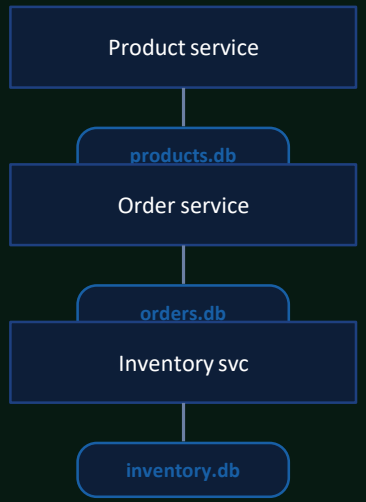


✗ Schema change breaks all services

✗ One slow query affects everyone

✗ Can't scale independently

✓ CORRECT



✓ Schema changes stay isolated

✓ Scale storage independently

✓ Each service picks its own DB type

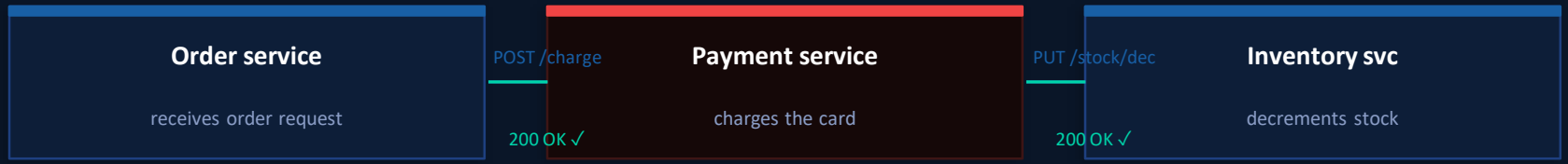
Rule to remember: if two services share a database, they are NOT really two services — they're still a monolith.

STEP 05 — SYNCHRONOUS CALLS

When a service needs an immediate answer

HTTP calls between services — ask and wait for the response.

Scenario: A user places an order. OrderService must confirm payment before confirming the order.



1 Order arrives

OrderService receives POST /orders from the gateway

2 Charge the card

Calls PaymentService synchronously — waits for confirmation

3 Update stock

Calls InventoryService — waits for stock to be decremented

4 Confirm order

Only now does OrderService return 200 OK to the client

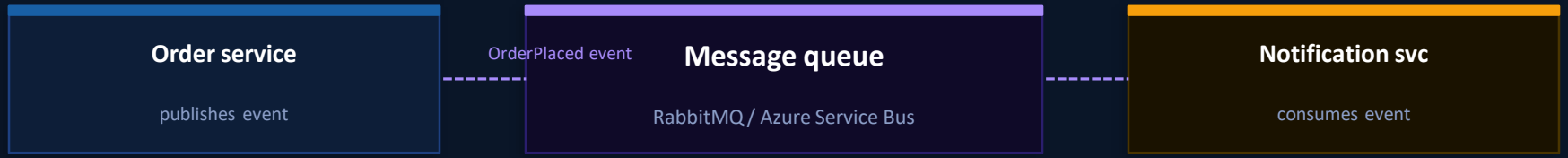
⚠ Risk: if PaymentService is down, OrderService fails too. This is the downside of sync calls — tight dependency.

STEP 06 — ASYNCHRONOUS EVENTS

Fire and forget

OrderService places an order — then moves on. It doesn't wait.

Scenario: Order is confirmed. An email should be sent. Does OrderService really need to wait for that email?



✓ **OrderService finishes instantly** — Doesn't wait for the email to send — response goes back to user immediately

✓ **NotificationService can be down** — Event stays in the queue. When it comes back up, it processes the message

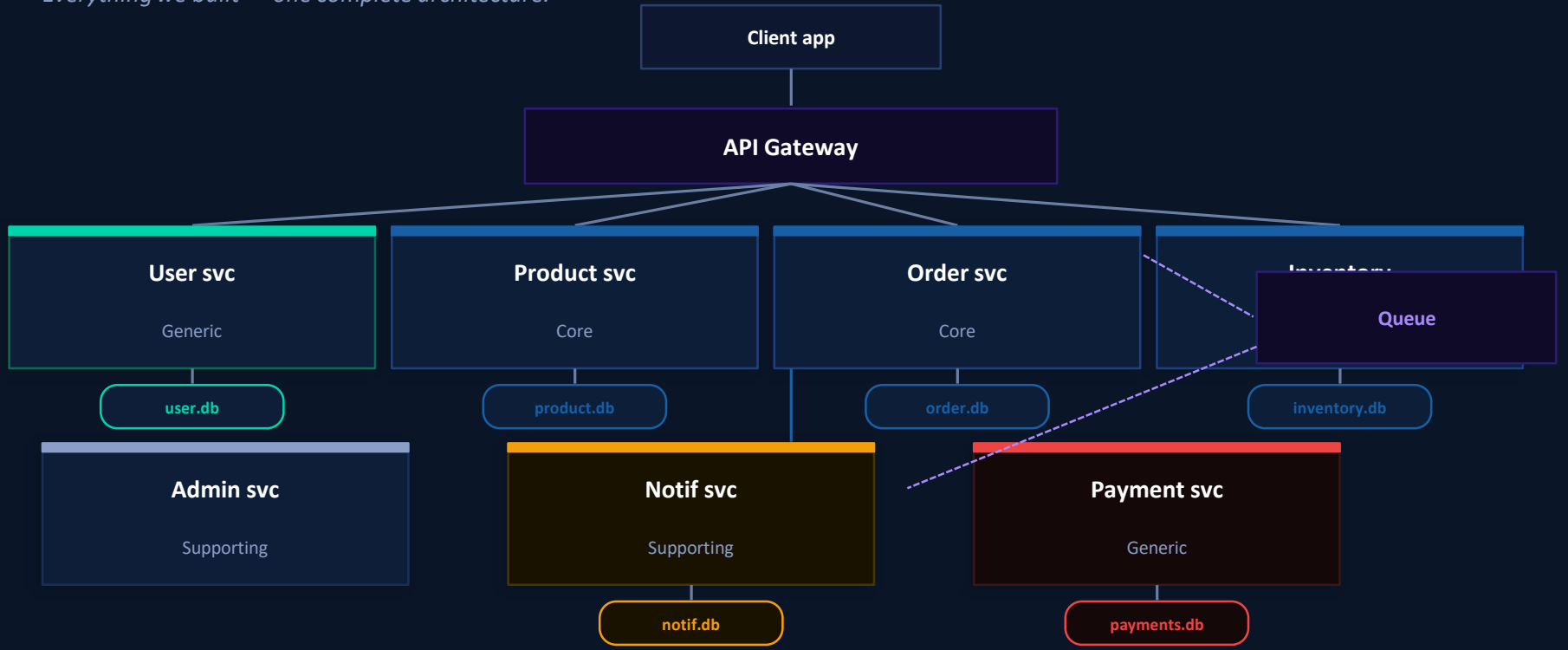
✓ **Easy to add consumers** — Want SMS too? Add an SMS service that also listens — no change to OrderService

Discussion: Which of these should be sync vs async? Login? Payment? Sending email? Updating inventory?

STEP 07 — THE FULL PICTURE

Now put it all together

Everything we built — one complete architecture.



CLASS DISCUSSION

Now let's talk about it

No wrong answers — your reasoning matters more than the result.

Q1 **Why does the client only know ONE address?**

💡 Hint: Think about what happens without a gateway — would the client need to know all 7 service addresses?

Q2 **UserService is Generic — so why build it at all?**

💡 Hint: We still need a service that wraps Auth0, maps users to our system, and stores profile data.

Q3 **Why is payment sync but notification async?**

💡 Hint: What happens if payment fails vs email fails? Which failure is acceptable to the user?

Q4 **What happens if the queue goes down?**

💡 Hint: Events are lost? Or queued? How does RabbitMQ handle this — and why does it matter for notifications?

Q5 **OrderService calls InventoryService via HTTP. What's the risk?**

💡 Hint: If inventory is slow, what happens to the order? How could you solve this?

Q6 **Why does AdminService have no database?**

💡 Hint: It reads data from all other services — why is that OK? When would it NOT be OK?

R E C A P

What we just built

Seven concepts — one complete architecture.

01 Client

Sends one HTTP request to one URL — knows nothing about the services behind it.

02 API Gateway

The single front door. Routes requests, validates auth, enforces rate limits.

03 Services

Each owns one domain. Core = business logic. Supporting = operational. Generic = buy it.

04 Databases

One database per service. No sharing. Ever. This is what makes true isolation possible.

05 Sync calls

HTTP between services when you need an immediate answer. Use sparingly — creates dependency.

06 Async events

Publish to a queue when you don't need to wait. Decouples services, improves resilience.

Next: Resilience, Fault Tolerance & Deployment →